

КОМАНДНЫЙ ПРОЕКТ

NetAnalysis

Анализ структуры и приближённых расстояний
в больших графах

Группа 25Б72-мм

Команда №3

СПбГУ

2026

Постановка задачи

Проблема: Графы социальных сетей содержат миллионы вершин и сотни миллионов рёбер. Точный расчёт кратчайших путей (APSP) непрактичен.

Три цели проекта

1. Разработать инструмент для **структурного анализа графов** — связность, кластеризация, устойчивость
2. Реализовать и сравнить **Landmark-алгоритмы оценки расстояний** (Basic LM, BFS LM) с 3 стратегиями выбора ориентиров
3. Провести эксперименты на **13 датасетах** — от 889 до ~3M вершин

Ключевые исследовательские вопросы

- Как устроена **топология реальных сетей**?
- Каков **trade-off между скоростью и точностью** landmark-алгоритмов?
- Какая **стратегия выбора ориентиров** оптимальна?

Архитектура

Архитектура NetAnalysys

Технологический стек:

Rust (stable 1.88-1.95)

crossterm

tokio

rayon

plotters

Модули

- `graph/` — списки смежности, маппинг ID, степени
- `parser/` — .txt / .csv / .mtx, автоопределение directed/undirected
- `analysis/` — 6 подмодулей: связность → степени → диаметр → треугольники → кластеризация → устойчивость
- `landmarks/` — Basic LM + BFS LM, 3 стратегии выбора

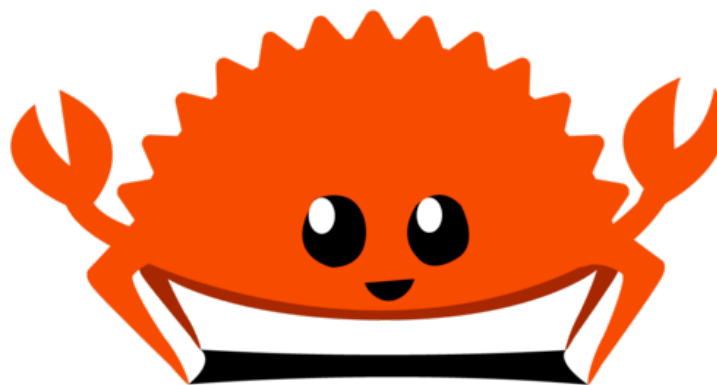
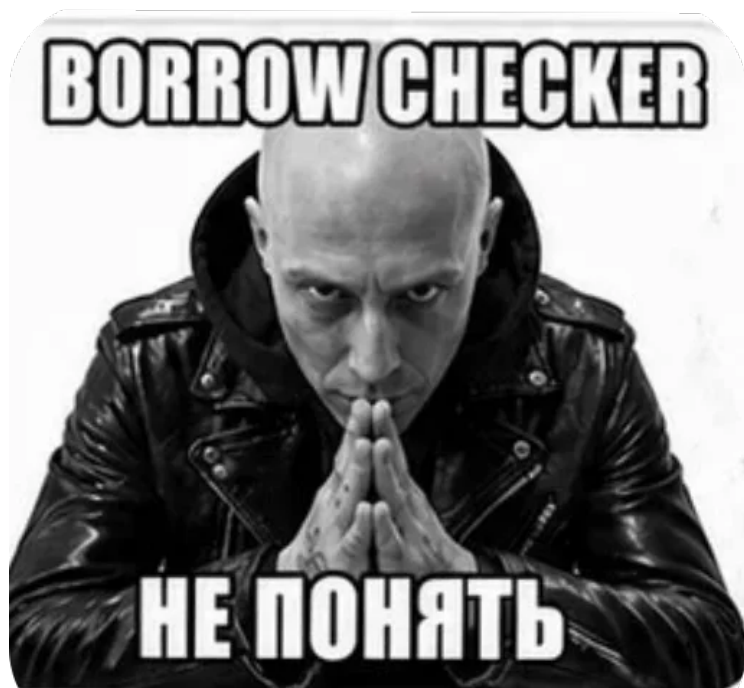
Интерфейс

- `ui/` — TUI-меню, файловый браузер
- `interactive_landmarks/` — запрос расстояний, accuracy- и speed-бенчмарки

Pipeline обработки

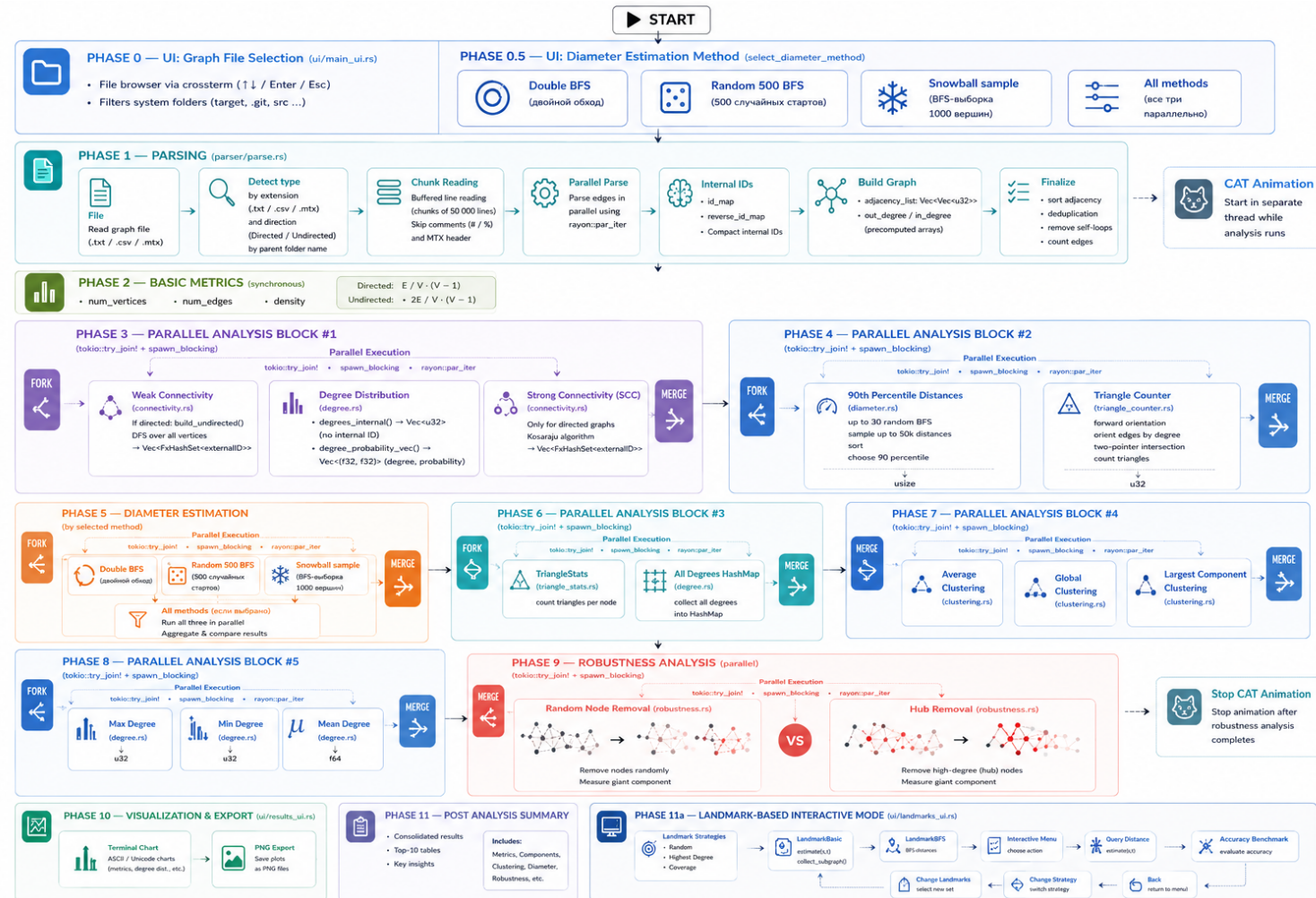
выбор датасета → парсинг → параллельный подсчёт метрик → вывод + PNG → интерактивный режим

Стек



Выбор очевиден

Pipeline обработки



Окружение

Компонент	Характеристика
Ноутбук	ASUS Vivobook 16X
CPU	AMD Ryzen 7 7730U (8 ядер / 16 потоков, 2.0–4.5 GHz)
GPU	Radeon Vega 8 (integrated)
RAM	16 GB DDR4
ОС	Linux
Язык / Бэкенд	Rust (stable), tokio + rayon

Все тесты проводились на одном устройстве при стандартных условиях. Длительность замеров фиксировалась через ручное логирование в отдельный файл `performance.log`.

Ключевые выводы по производительности:

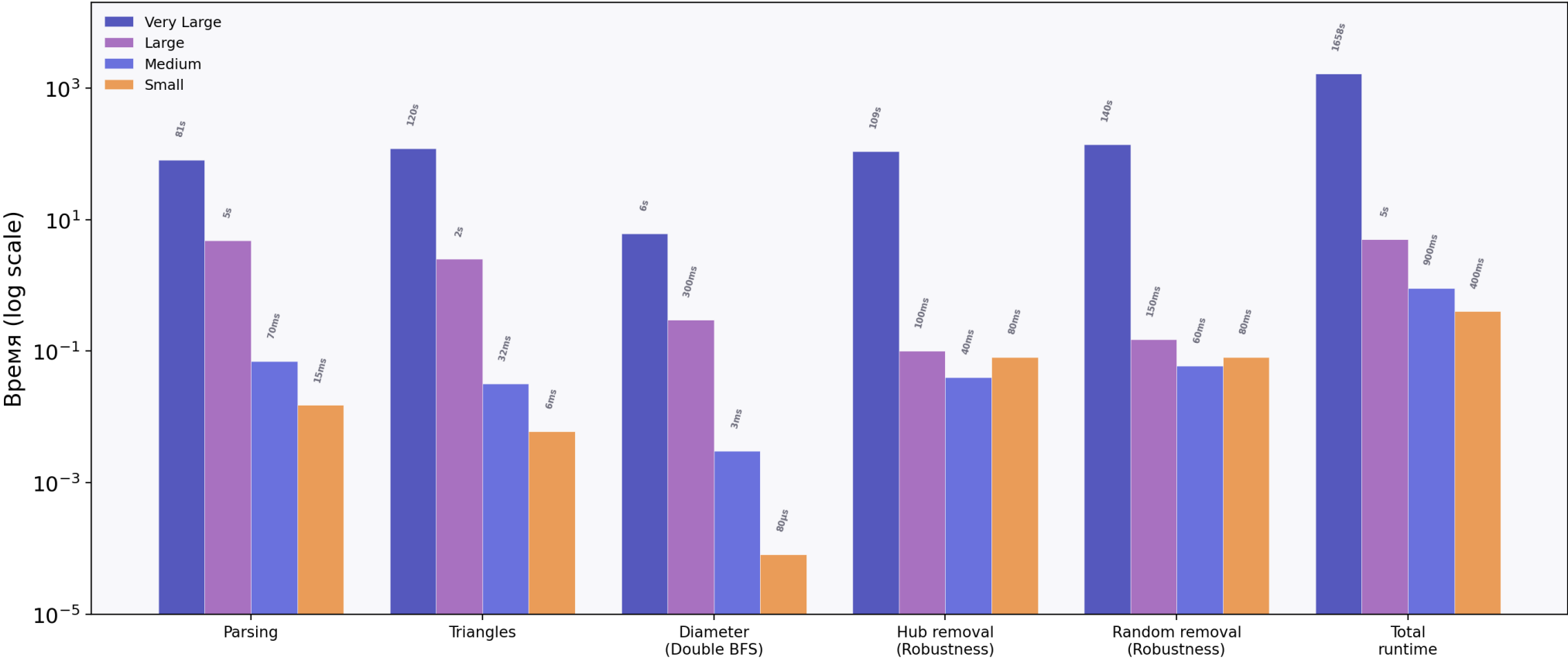
- Треугольники и Random 500 BFS — самые дорогие операции: до 120 с и **20 мин** на orkut
- Double BFS на 3 порядка быстрее Random 500 BFS: 6 с vs 20 мин (**×200**)
- На малых графах (< 10K вершин) все pipeline-метрики считаются за доли секунды

Производительность: время выполнения этапов

Этап	Very Large (orkut) 3.07M V, 117M E	Large 876K–3.2M V	Medium 18K–326K V	Small 889–7K V
Парсинг (parse_file)	81 s	4.8 s	70 ms	15 ms
Слабая связность (WCC)	2.5 s	0.2 s	4 ms	0.3 ms
Сильная связность (SCC)	—	0.7 s	1.6 ms	0.2 ms
Степени (degree distribution)	14 ms	3 ms	0.1 ms	0.02 ms
Треугольники (triangle count)	120 s	2.5 s	32 ms	6 ms
Кластеризация (clustering coeff.)	9 ms	2 ms	0.2 ms	0.04 ms
Double BFS (диаметр)	6.1 s	0.3 s	3 ms	80 µs
P90 расстояний (percentile)	30 s	1 s	12 ms	0.3 ms
Hub removal (robustness)	109 s	0.1 s	40 ms	80 ms
Random removal (robustness)	140 s	0.15 s	60 ms	80 ms
Random 500 BFS (отд. замер)	~20 min	5–10 s	0.5 s	35 ms
Total runtime	~28 min	~5 s	~0.9 s	~0.4 s

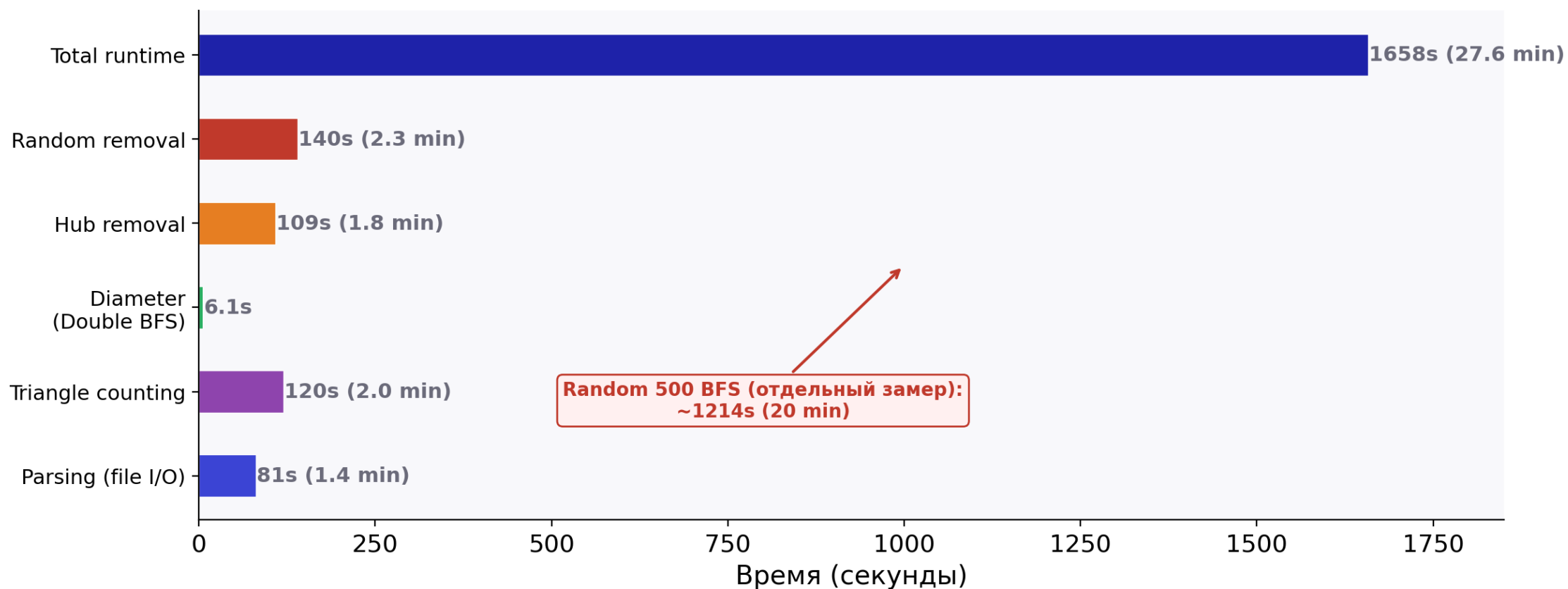
Сводка времени обработки по категориям

Время обработки графов: сводка по категориям (log scale)



Поэтапное время: Very Large граф (com-orkut)

Поэтапное время: Very Large (com-orkut, 3.07М вершин, 117М рёбер)



Датасеты

13 графов, 3 категории — широкий охват предметных областей

Категория	Кол-во	Диапазон вершин	Диапазон рёбер	Примеры
Directed	5	7K – 876K	2.9K – 5.1M	web-Google, web-Stanford, wiki-Vote, soc-wiki-Vote, web-NotreDame
Undirected	5	889 – 1.7M	29K – 15.2M	ca-coauthors-dblp, CA-GrQc, CA-AstroPh, musae-git-edges, ca-HepTh
Large	3	1.1M – 3.07M	3M – 117M	com-youtube, com-orkut, vk

От коллабораций учёных → веб-графы → социальные сети → мессенджеры.

Результаты анализа

Pipeline анализа и вычисляемые метрики

Блок 1. Базовая статистика

- $|V|$, $|E|$, плотность
- Weak / Strong компоненты
- LWCC / LSCC

Блок 4. Треугольники и кластеризация

- Полный подсчёт треугольников
- Средний коэффициент
- Глобальный коэффициент

Блок 2. Степени

- min / max / avg degree
- Распределение степеней
- Regular + log-log шкалы

Блок 5. Устойчивость

- Random removal (5%–100%)
- Degree-targeted removal
- Доля вершин в giant component

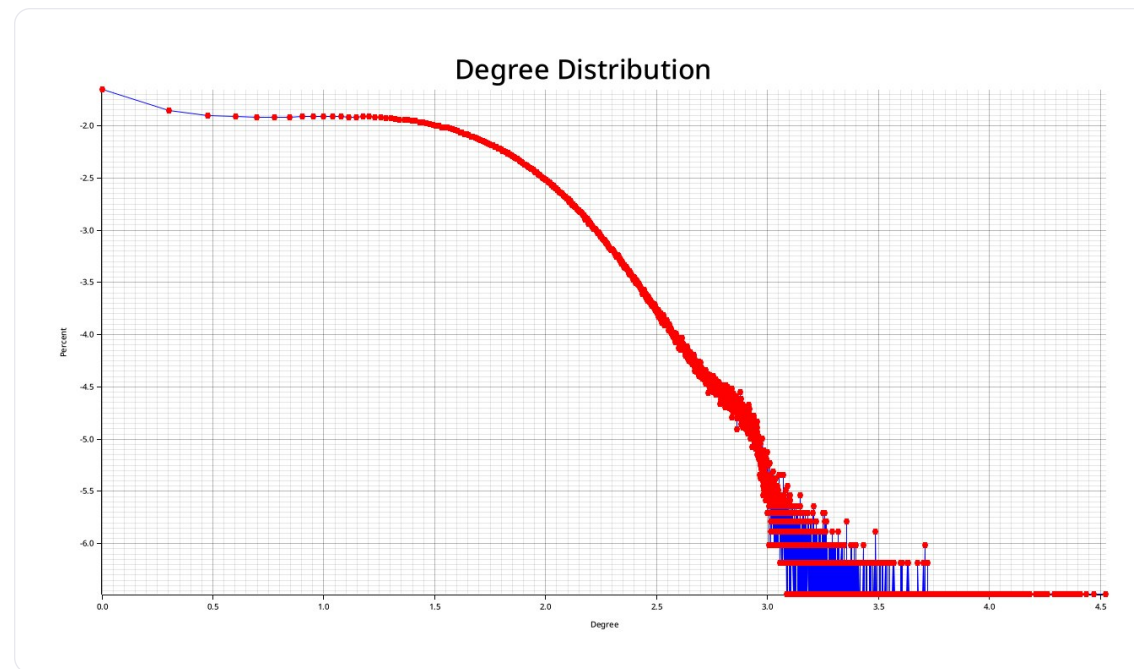
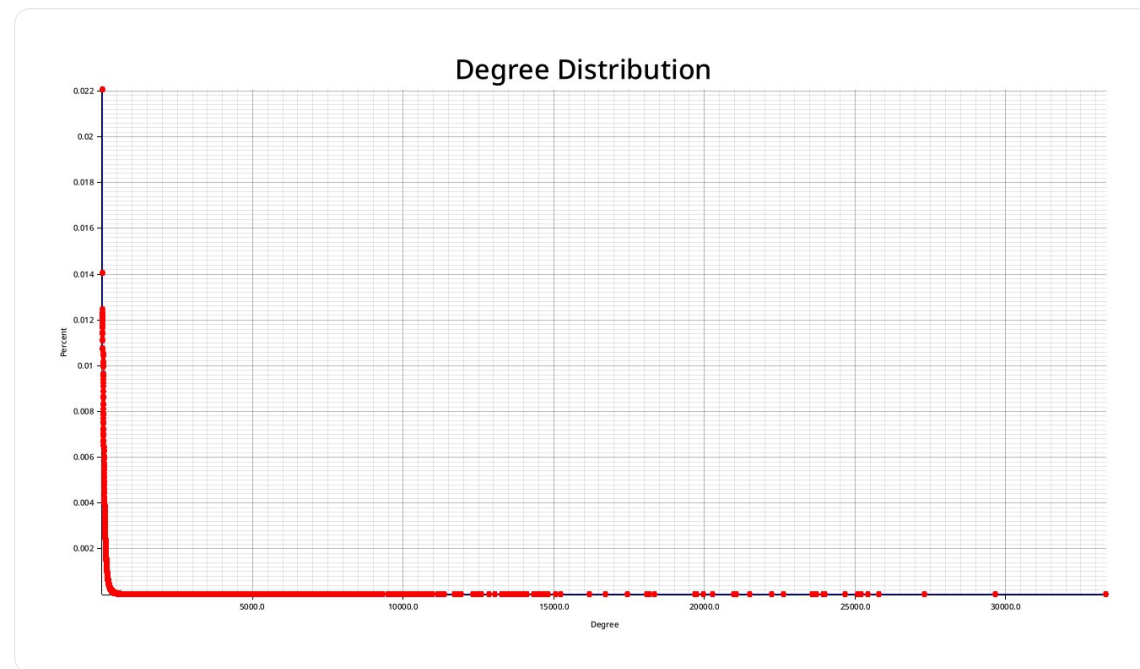
Блок 3. Диаметр

- Double BFS
- Random 500 BFS
- Snowball sampling
- 90-й перцентиль

Визуализация

- PNG-графики распределений
- Терминальные графики
- Интерактивные запросы

Распределение степеней



Базовая статистика графов

Все графы крайне разрежены: density $\sim 10^{-5} \dots 10^{-3}$

Параметр	Undirected (mean $\pm \sigma$)	Directed (mean $\pm \sigma$)	Large/youtube	Large/orkut
Density	0.00067 $\pm$ 0.00050	0.00151 $\pm$ 0.00171	0.000005	0.000025
LWCC	0.937 $\pm$ 0.098	0.975 $\pm$ 0.046	1.000	1.000
LSCC	—	0.221 $\pm$ 0.224	—	—
Avg Degree	24.59 $\pm$ 22.17	15.28 $\pm$ 10.15	5.27	76.28
Diameter (DBFS)	16.25 $\pm$ 5.12	12.0 $\pm$ 21.4	24	10

Ключевой инсайт: у directed-графов слабая связность высока (**LWCC ~ 0.97**), но сильная — крайне низкая (**LSCC ~ 0.22**). Разрыв LWCC – LSCC ≈ 0.75 .

Outlier: soc-wiki-Vote — LSCC ≈ 0.001 (сильная компонента практически отсутствует).

Кластеризация и треугольники

Диапазон clustering coefficient — от **0.08** до **0.80**.

Граф	Треугольники	Avg CC	Global CC	Тип
ca-coauthors-dblp	Гигантское	0.80	0.08 (при 1.7М вершин)	Плотная сообщественная структура
com-orkut	627M	0.1666	0.041 (при 3М вершин)	Плотный, хабово-кластеризованный
Wiki-vote / soc-wiki-Vote	Умеренное	0.15–0.25	Низкий	Локальные кластеры без глобального замыкания
web-NotreDame / youtube	Низкое	Низкий	Низкий	Разреженная backbone-структура

Clustering — маркер типа сети: высокий → community-heavy, низкий → hub-dominated.

Устойчивость к удалению вершин

Два сценария: **random removal** vs **degree-targeted removal**

Граф	Random (50% удалено)	Targeted (50% удалено)	Характер
ca-coauthors	Giant comp. сохранена	Медленная деградация, затем резкий обвал	Умеренная hub-зависимость
musae-git-edges	Giant comp. сохранена	Быстрый распад	Выраженная hub-архитектура
web-Stanford	Giant comp. сохранена	Резкий пороговый распад	Сильная hub-архитектура
web-NotreDame	Giant comp. сохранена	Экстремально быстрый коллапс	Экстремальный hub-dominance

Вывод: все графы — hub-driven / scale-free. Случайное удаление разрушает медленно, удаление хабов — резкий перколяционный порог.

Large-группа: неоднородность внутри категории

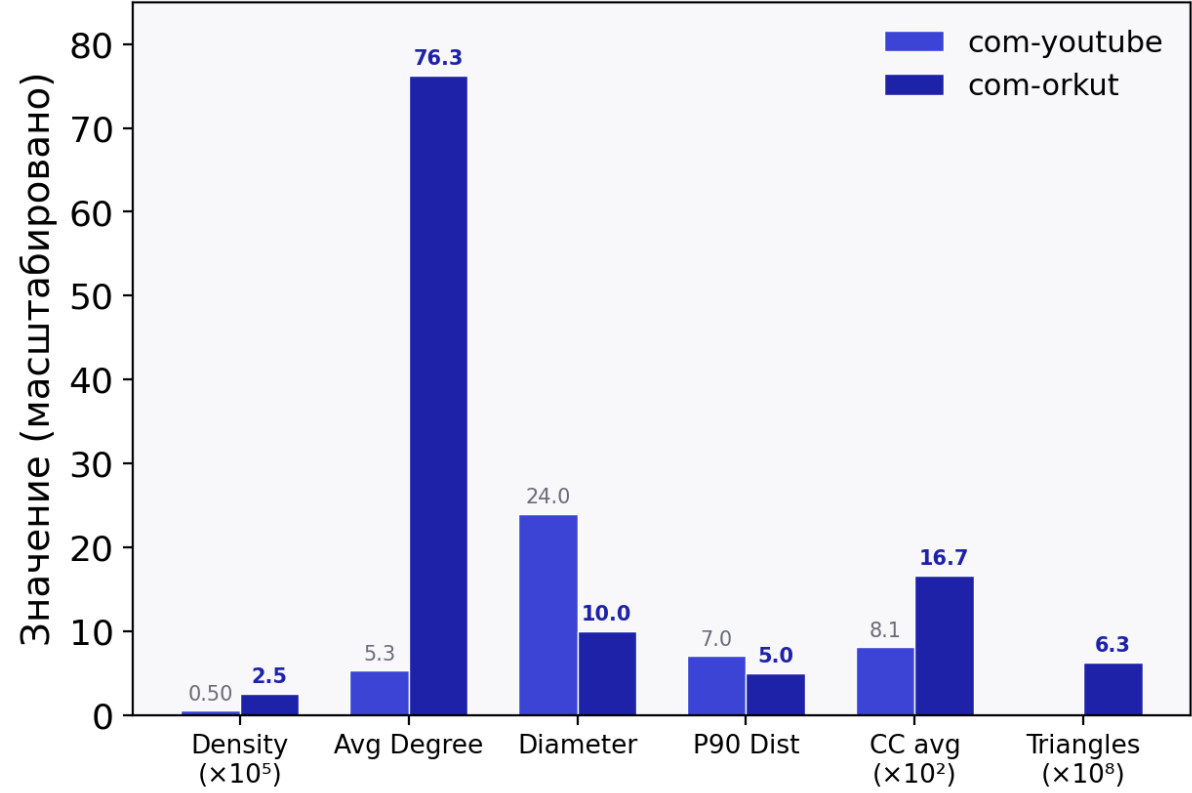
Large — не «один тип», а **контрастная пара**: два очень разных режима генерации сети

Параметр	com-youtube	com-orkut	Δ
V	1.13M	3.07M	×2.7
E	2.99M	117.19M	×39
Density	5×10^{-6}	2.5×10^{-5}	×5
Avg Degree	5.27	76.28	×14
Diameter (DBFS)	24	10	−58%
P90 distance	7	5	−29%
CC avg	0.0808	0.1666	×2
Треугольники	~0	627M	Взрыв
k_max	—	33 313	Гигантский хаб

Ключевой вывод: масштаб не ведёт к автоматическому разрежению. Orkut больше youtube, но **плотнее в 5 раз, короче по путям на 58% и сильнее кластеризован в 2 раза**. Large-категория смешивает принципиально разные структурные режимы.

Large-группа: контраст youtube vs orkut

Large-группа: контрастная пара



Параметр	youtube → orkut	Δ
V	1.13M → 3.07M	$\times 2.7$
density	$5 \times 10^{-6} \rightarrow 2.5 \times 10^{-5}$	$\times 5$
avg degree	5.27 → 76.28	$\times 14$
diameter	24 → 10	-58%
p90 distance	7 → 5	-29%
CC avg	0.081 → 0.167	$\times 2$
triangles	$\sim 0 \rightarrow 627\text{M}$	взрыв
k_max	$\rightarrow 33\ 313$	гигантский хаб

Масштаб не ведёт к разрежению:
orkut больше youtube, но плотнее,
короче по путям и сильнее
кластеризован

Landmark-алгоритмы

Landmark-алгоритмы: постановка задачи

Зачем нужны landmarks?

Точный BFS на графе с **3М вершин и 117М рёбер** — **1.26–1.57 с** на один запрос. Для тысячи запросов — часы.

Идея: выбрать **k ориентиров** (landmarks), предпосчитать расстояния от них, оценивать расстояние между любыми двумя вершинами через ориентиры за $O(k)$.

Два алгоритма

- **Basic LM:** $d(s,t) \approx \min_l [d(s,l) + d(l,t)]$
- **BFS LM:** сохраняем BFS-деревья, на запросе строим подграф пересечения путей $\rightarrow$ BFS

Три стратегии выбора

1. **Random** — случайный выбор
2. **Highest-degree** — вершины с максимальной степенью
3. **Farthest-first coverage** — максимальное геодезическое покрытие

Landmark: архитектура реализации

LandmarkSelection (enum)

- └ Random
- └ HighestDegree
- └ Coverage (farthest-first)

LandmarkBasic LandmarkBFS

- └ precompute() └ precompute()
 - └ BFS от каждого LM └ BFS + parent tree
- └ estimate(s, t) └ estimate(s, t)
- └ min(d(s,LM) + collect_subgraph(s→LM + t→LM)
d(LM, t)) → BFS на подграфе

Характеристика	Basic LM	BFS LM
Precomputation	$O(k \cdot (V+E))$	$O(k \cdot (V+E))$
Запрос	$O(k)$	$O(\text{subgraph})$
Точность	Ниже	Выше
Память	$k \cdot V $ расстояний	$k \cdot V $ расстояний + parent trees

Landmark: скорость на большом графе (~1.5 GB)

Условия: ~10 и ~100 landmarks, 50 случайных пар, 3 стратегии

Метод	10 LM	100 LM	Ускорение vs Exact
Exact BFS	1.26–1.43 s	1.46–1.57 s	1×
Basic LM	3.1–3.3 μ s	0.23–1.61 ms	×400 000 → ×1 000
BFS LM	11.6–11.9 ms	15.0–18.0 ms	×100 → ×80

- Basic LM ускоряет exact BFS в ~400 000 раз (10 LM) и в ~1 000 раз (100 LM)
- BFS LM — компромисс: ×80–100 быстрее exact, но точнее Basic
- Рост LM с 10 до 100 → Basic LM замедляется в ~500 раз — trade-off speed vs quality

Landmark: скорость на малом графе (Wiki-Vote)

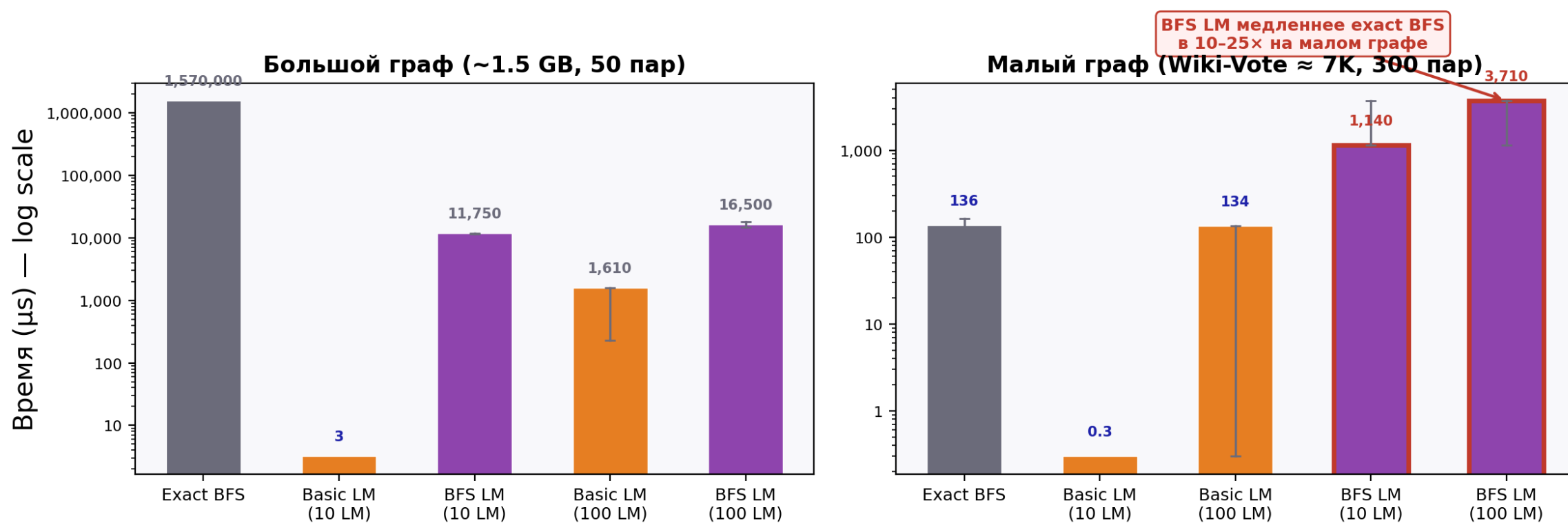
Условия: **Wiki-Vote** (~7K вершин), 300 случайных пар

Метод	Среднее время	vs Exact
Exact BFS	107–165 μ s	1× (эталон)
Basic LM	0.3–134 μ s	×1.5–500 (быстрее)
BFS LM	1.14–3.71 ms	×10–25 медленнее exact

Критический вывод: на маленьких графах BFS LM теряет смысл как ускоритель — overhead precomputation + построение подграфа **дороже**, чем просто запустить exact BFS. Basic LM при этом всё ещё выигрывает.

Диаграммы по скорости

Landmark: скорость (log scale)



Landmark: точность

Большой граф (~1.5 GB, 50 пар)

Параметр	Basic LM	BFS LM
Exact-match rate (10 LM)	0–26%	28–48%
Exact-match rate (100 LM)	0–58%	74–88%
Средняя ошибка	0.46–4.14	0.00–0.26

Wiki-Vote (≈7K, 300 пар)

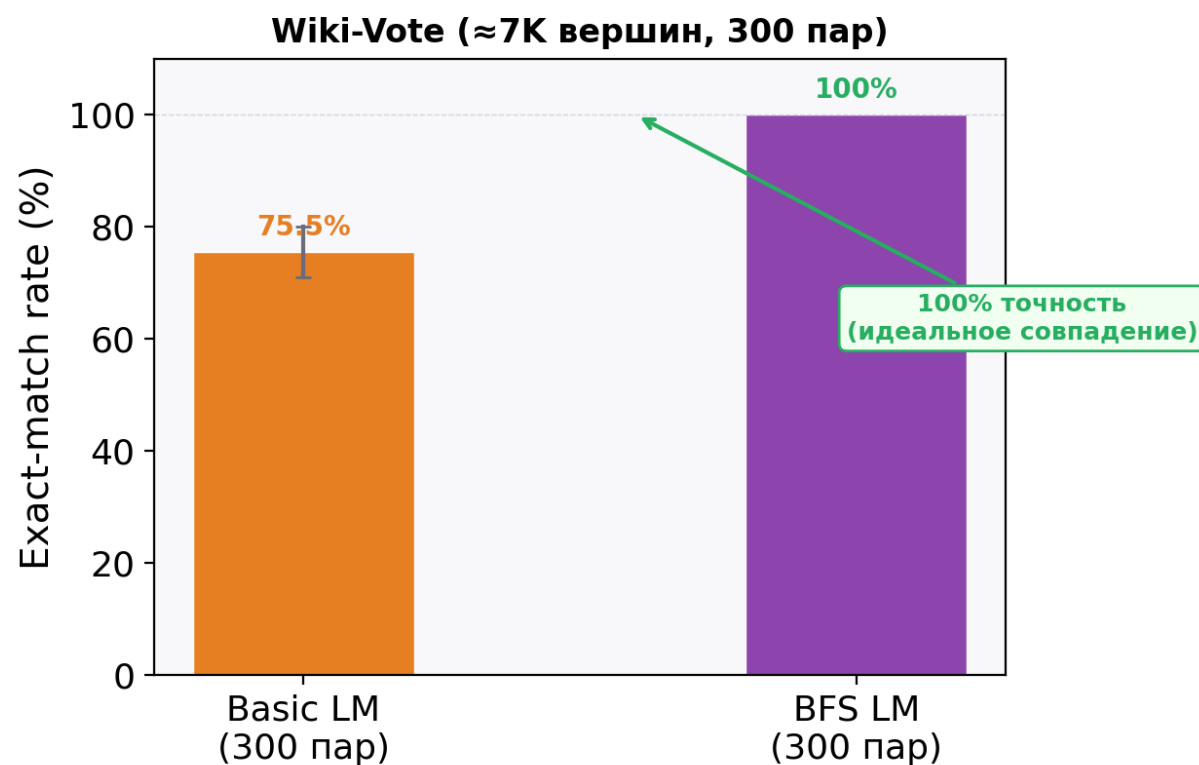
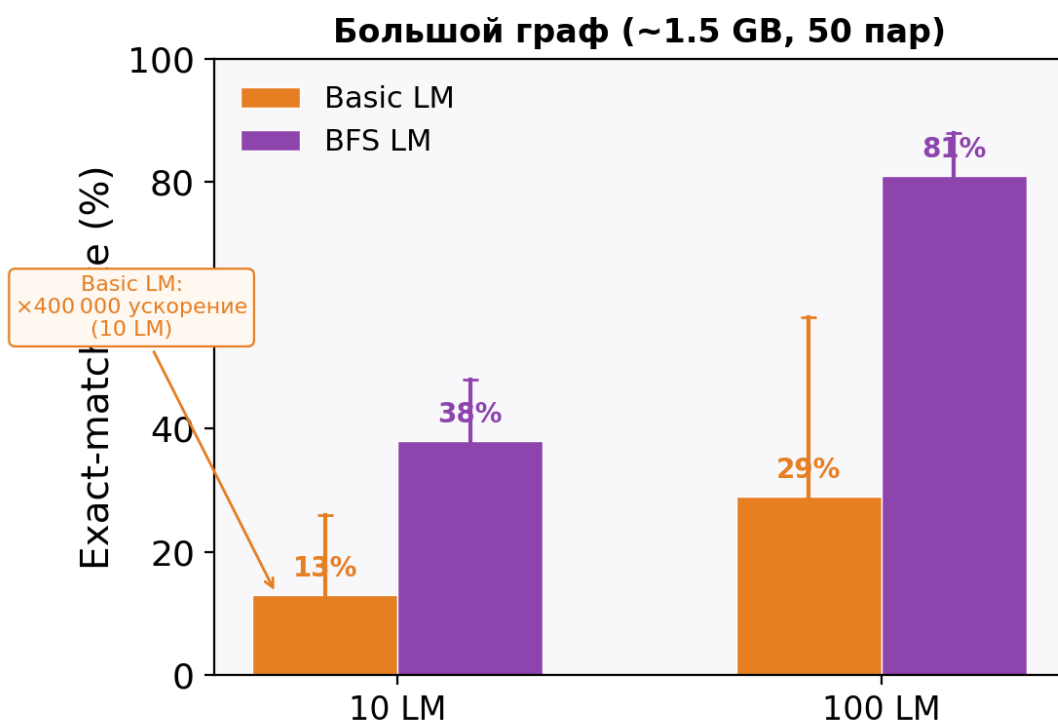
Параметр	Basic LM	BFS LM
Exact-match rate	71–80%	100%
Средняя ошибка	Малая	0 (идеально)

Закономерности:

- BFS LM стабильно точнее Basic LM на всех размерах
- Увеличение LM с 10 → 100 резко повышает точность (BFS LM: 28–48% → 74–88%)
- На малом графе BFS LM достигает 100% совпадения с exact — идеальная точность
- Средняя ошибка Basic LM (0.46–4.14) на порядок выше, чем BFS LM (0.00–0.26)

Точность Landmark-алгоритмов

Landmark: точность (exact-match rate)



Landmark: влияние стратегий выбора

На большом графе (~1.5 GB)

- **Random:** хороший baseline, конкурентен для BFS LM
- **Highest-degree:** самая **сбалансированная** для Basic LM — лучший exact-match rate, ниже ошибка
- **Farthest-first:** не универсальный лидер — иногда уступает degree по accuracy

На малом графе (Wiki-Vote)

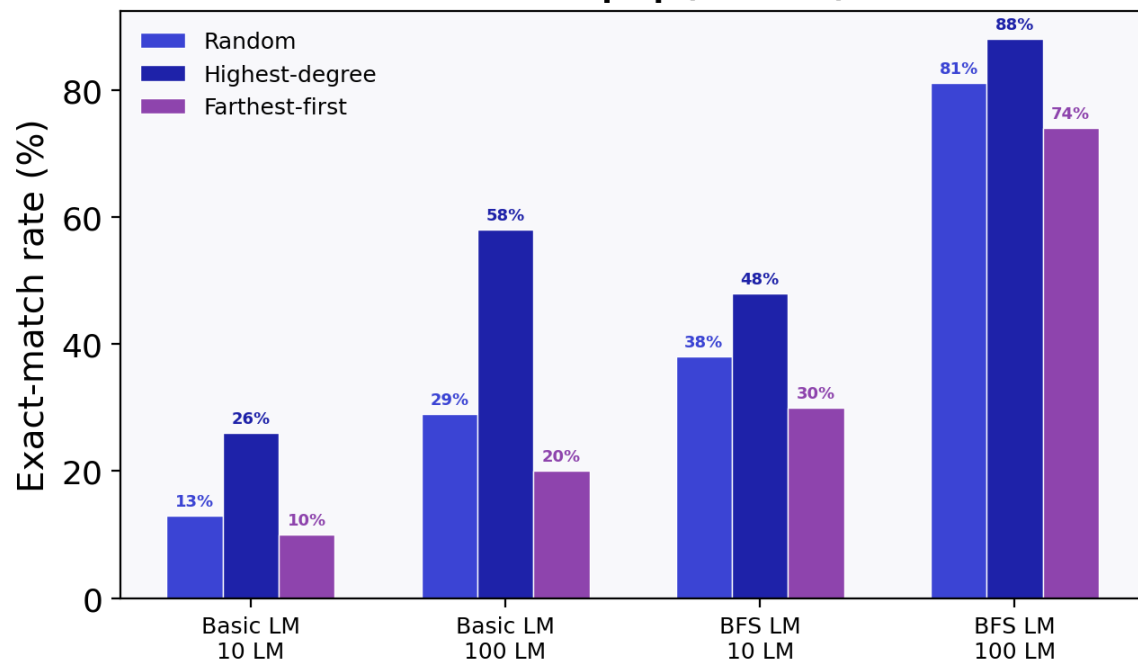
- **Highest-degree:** стабильна, хороша по accuracy
- **Farthest-first:** лучшая **latency** для Basic LM, но по точности не всегда выигрывает
- **Random:** приемлема, но не оптимальна

Статистическая оговорка: на 50 парах разница 44% vs 48% — шум. Разница 0% vs 58% или 28% vs 85% — реальный эффект. На 300 парах выводы надёжнее.

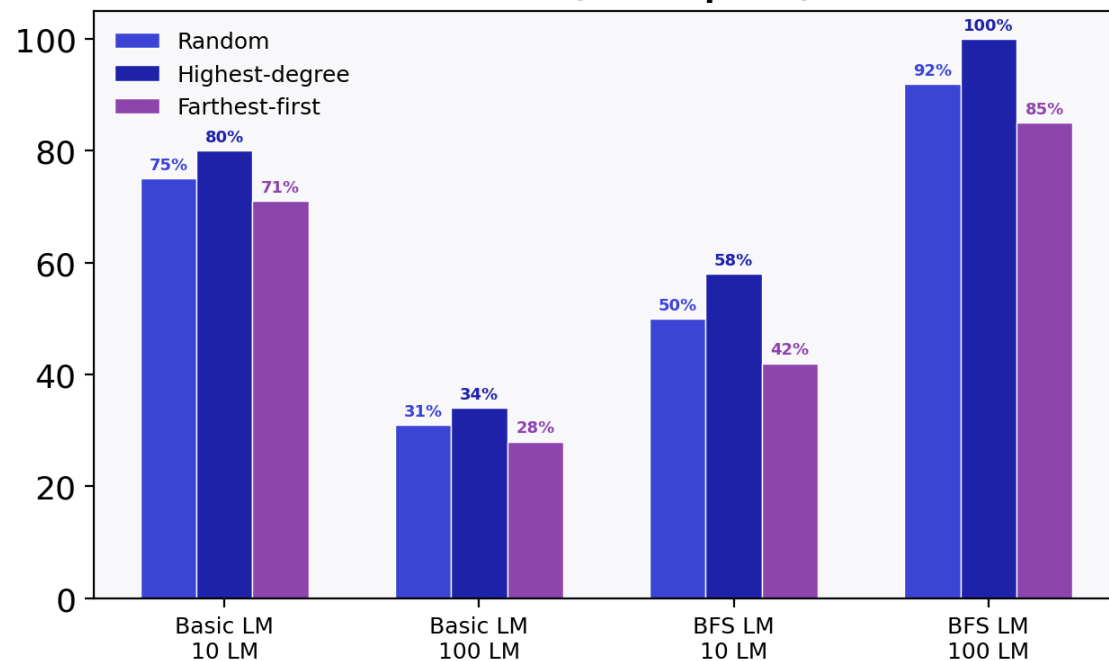
Влияние стратегий выбора ориентиров

Landmark: влияние стратегий выбора ориентиров

Большой граф (~1.5 GB)



Wiki-Vote (≈7K вершин)



Практические рекомендации

Если ваша цель	Алгоритм	Стратегия	k	Ожидаемый результат
Максимальная скорость	Basic LM	Random / Farthest-first	10	×400 000 ускорение, точность 0–26%
Максимальная точность	BFS LM	Highest-degree	100	до 88% exact-match, ×80 ускорение
Баланс speed/quality	BFS LM	Highest-degree	30–50	×200 ускорение, 60–70% точности
Маленький граф (< 50K)	Exact BFS	—	—	дешевле, чем BFS LM
Low-latency на малом графе	Basic LM	Farthest-first	10	микросекунды, accuracy ~71–80%

Правило большого пальца: Basic LM → взрывная скорость, умеренная точность. BFS LM → высокая точность, хорошее ускорение (но не на малых графах). Degree-based — самая устойчивая стратегия в большинстве сценариев.

Заключение

Заключение

Что сделано

- Разработано **Rust TUI-приложение** для полного цикла анализа графов
- Реализованы **3 метода оценки диаметра** + **2 landmark-алгоритма** + **3 стратегии**
- Проведены эксперименты на **13 датасетах**: от 889 до **3.07М вершин**

Перспективы

CI/CD

Тестовое покрытие

Библиотечный crate

WebAssembly

Ключевые выводы

1. Реальные графы — **scale-free** с hub-архитектурой; directed — разрыв LWCC/LSCC ~ 0.75
2. Large-группа **неоднородна**: youtube (diam=24) vs orkut (diam=10, CC $\times 2$)
3. **Basic LM**: до $4 \times 10^5 \times$ ускорение, точность 0–58%
4. **BFS LM**: до **88%** точности при 100 LM, $\times 80$ –100
5. **Degree-based** — самая устойчивая стратегия; на малых графах — exact BFS или Basic LM

NetAnalisisys

Мороз Владислав • Овечкин Андрей
Санкт-Петербургский государственный университет • 2026

© 2026

Creative Commons Attribution 4.0 International
(CC BY 4.0)

Research • Engineering • Open Knowledge

creativecommons.org/licenses/by/4.0